

Parte 1 - Infraestrutura

Para as questões a seguir, você deverá executar códigos em um notebook Jupyter, rodando em ambiente local, certifique-se que:

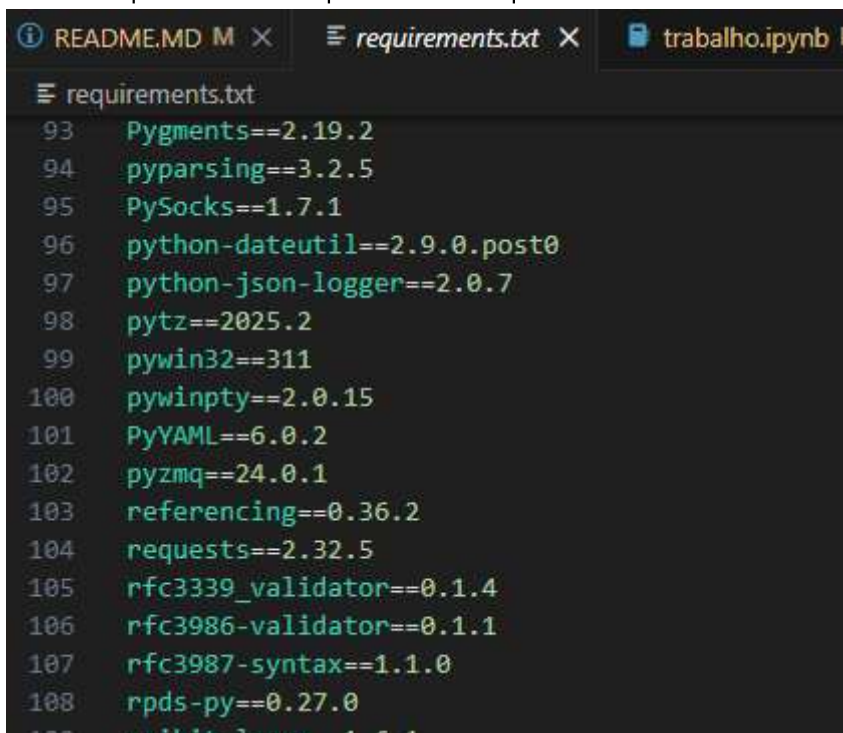
1. Você está rodando em Python 3.9+

```
(py39) C:\Users\alan echer\Documents\25E4-infnet-algoritmos-de-inteligencia-artificial-para-clusterizacao>python --version
Python 3.9.23
(py39) C:\Users\alan echer\Documents\25E4-infnet-algoritmos-de-inteligencia-artificial-para-clusterizacao>
```

2. Você está usando um ambiente virtual: Virtualenv ou Anaconda

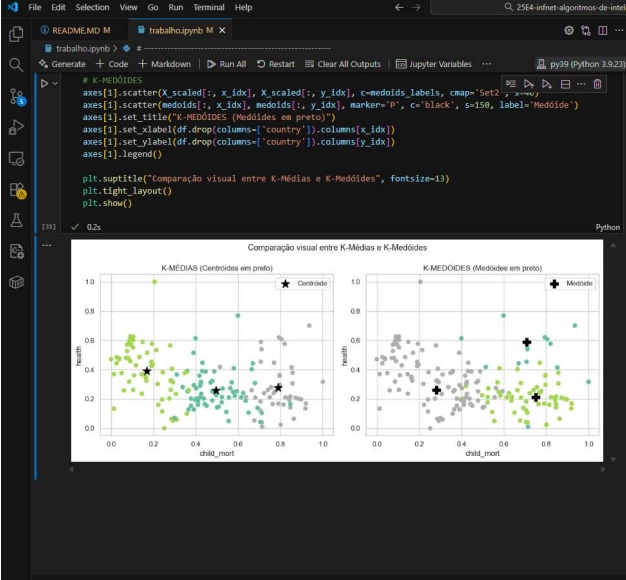
```
(base) C:\Users\alan echer\Documents\25E4-infnet-algoritmos-de-inteligencia-artificial-para-clusterizacao>conda activate py39
(py39) C:\Users\alan echer\Documents\25E4-infnet-algoritmos-de-inteligencia-artificial-para-clusterizacao>
```

3. Todas as bibliotecas usadas nesse exercício estão instaladas em um ambiente virtual específico anaconda - py39
4. Gere um arquivo de requerimentos (requirements.txt) com os pacotes necessários. É necessário se certificar que a versão do pacote está disponibilizada.



```
requirements.txt
93 Pygments==2.19.2
94 pyparsing==3.2.5
95 PySocks==1.7.1
96 python-dateutil==2.9.0.post0
97 python-json-logger==2.0.7
98 pytz==2025.2
99 pywin32==311
100 pywinpty==2.0.15
101 PyYAML==6.0.2
102 pyzmq==24.0.1
103 referencing==0.36.2
104 requests==2.32.5
105 rfc3339_validator==0.1.4
106 rfc3986_validator==0.1.1
107 rfc3987-syntax==1.1.0
108 rpds-py==0.27.0
109 scikit-learn==1.6.1
```

5. Tire um printscreen do ambiente que será usado rodando em sua máquina.



Parte 1 - Infraestrutura

Para as questões a seguir, você deverá executar códigos em um notebook Jupyter, rodando em ambiente local, certifique-se que:

1. Você está rodando em Python 3.9+
2. Você está usando um ambiente virtual: Virtualenv ou Anaconda
3. Todas as bibliotecas usadas nesse exercício estão instaladas em um ambiente virtual específico anaconda - py39
4. Gere um arquivo de requerimentos (requirements.txt) com os pacotes necessários. É necessário se certificar que a versão do pacote está disponibilizada.
5. Tire um printscreen do ambiente que será usado rodando em sua máquina
6. Disponibilize os códigos gerados, assim como os artefatos acessórios (requirements.txt) e instruções em um repositório GIT público. (se isso não for feito, o diretório com esses arquivos deverá ser enviado compactado no moodle).

<https://github.com/echer/25E4-infnet-algoritmos-de-inteligencia-artificial-para-clusterizacao.git>

Parte 2 - Escolha de base de dados

Para as questões a seguir, usaremos uma base de dados e faremos a análise exploratória dos dados, antes da clusterização.

1. Baixe os dados disponibilizados na plataforma Kaggle sobre dados sócio-econômicos e de saúde que determinam o

6. Disponibilize os códigos gerados, assim como os artefatos acessórios (requirements.txt) e instruções em um repositório GIT público. (se isso não for feito, o diretório com esses arquivos deverá ser enviado compactado no moodle).

<https://github.com/echer/25E4-infnet-validacao-de-modelos-de-inteligencia-artificial-para-clusterizacao.git>

Parte 2 - Escolha de base de dados

Para as questões a seguir, usaremos uma base de dados e faremos a análise exploratória dos dados, antes da clusterização.

1. Escolha uma base de dados para realizar o trabalho. Essa base será usada em um problema de clusterização.

Os dados foram disponibilizados na plataforma Kaggle sobre dados sócio-econômicos e de saúde que determinam o índice de desenvolvimento de um país. Esses dados estão disponibilizados através do link: <https://www.kaggle.com/datasets/rohan0301/unsupervised-learning-on-country-data>

```
# PARTE 2 - 1 Baixe os dados disponibilizados na plataforma Kaggle sobre dados sócio-econômicos
# e de saúde que determinam o índice de desenvolvimento de um país.
# Esses dados estão disponibilizados através do link:
# https://www.kaggle.com/datasets/rohan0301/unsupervised-learning-on-country-data
path = kagglehub.dataset_download("rohan0301/unsupervised-learning-on-country-data")
```

2. Escreva a justificativa para a escolha de dados, dando sua motivação e objetivos.

A escolha do conjunto de dados sobre indicadores socioeconômicos de diferentes países foi motivada pela necessidade de compreender padrões globais de desenvolvimento e bem-estar utilizando variáveis quantitativas confiáveis. Esse tipo de informação é fundamental para análises comparativas, identificação de grupos semelhantes e suporte à tomada de decisão em políticas públicas, planejamento econômico e estudos internacionais.

3. Mostre através de gráficos a faixa dinâmica das variáveis que serão usadas nas tarefas de clusterização. Analise os resultados mostrados. O que deve ser feito com os dados antes da etapa de clusterização?

1. Não há valores nulos no conjunto de dados
2. Tem 167 países no total

3. Todas as variáveis são numéricas, exceto a coluna 'country'.
4. Foi observado grandes diferenças nas escalas, os dados possuem variação alta, presença de outliers e diferença de escala entre variáveis.

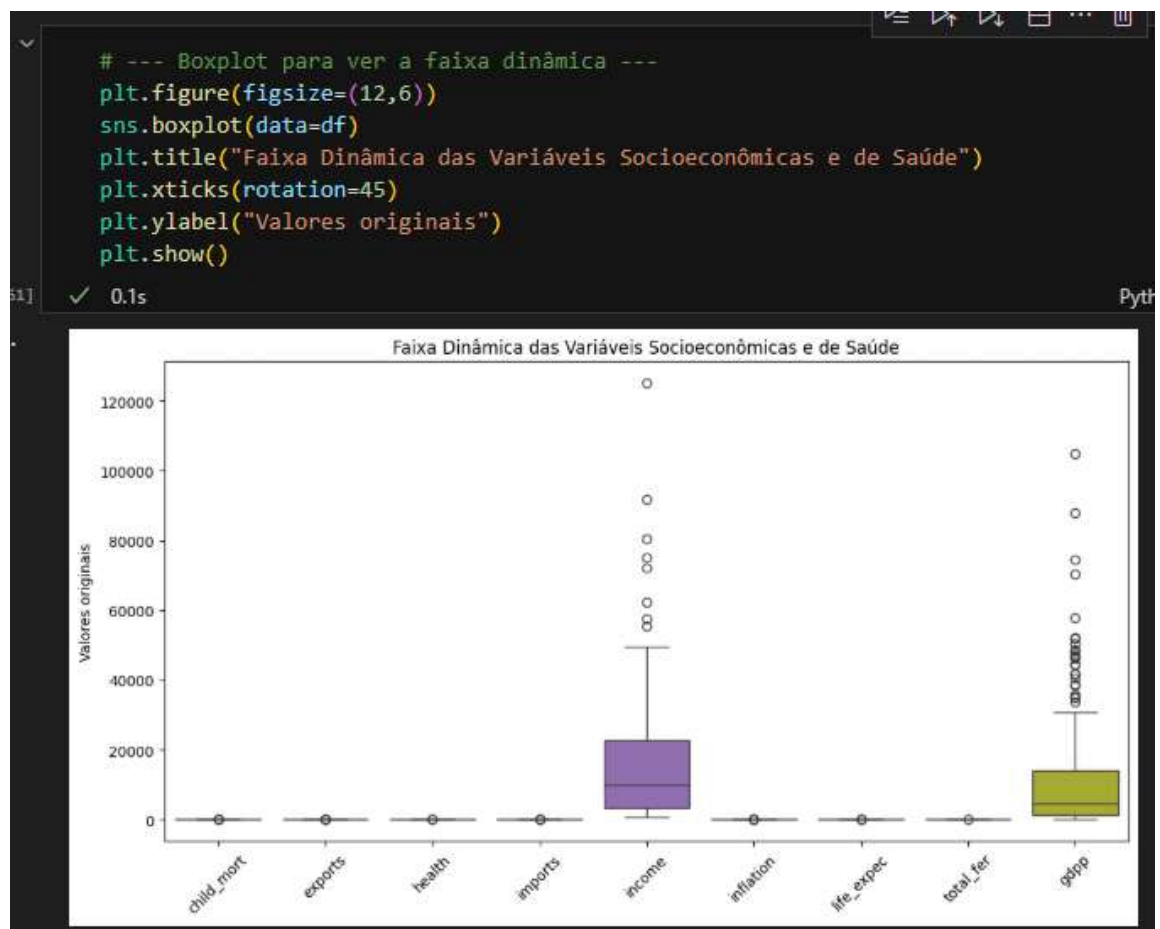
```
# Visualizando as primeiras linhas e informações básicas
print("Primeiras linhas do dataset:")
print(df.head())
print("\nInformações do dataset:")
print(df.info())
print("\nEstatísticas descritivas:")
print(df.describe())
print("\nValores nulos por coluna:")
print(df.isnull().sum())
```

142] ✓ 0.0s Python

```
-- Primeiras linhas do dataset:
      country  child_mort  exports  health  imports  income  \
0  Afghanistan      90.2     10.0    7.58     44.9    1610
1    Albania      16.6     28.0    6.55     48.6    9930
2    Algeria      27.3     38.4    4.17     31.4   12900
3    Angola     119.0     62.3    2.85     42.9    5900
4 Antigua and Barbuda    10.3     45.5    6.03     58.9   19100

      inflation  life_expec  total_fer  gdpp
0         9.44        56.2         5.82   553
1         4.49        76.3         1.65  4090
2        16.10        76.5         2.89  4460
3        22.40        60.1         6.16  3530
4         1.44        76.8         2.13 12200

Informações do dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 167 entries, 0 to 166
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   country    167 non-null    object
1   child_mort  167 non-null    float64
2   exports    167 non-null    float64
3   health     167 non-null    float64
...
life_expec    0
total_fer     0
gdpp          0
```



4. Realize o pré-processamento adequado dos dados. Descreva os passos necessários.

1. Remocao das colunas nao númericas.

```

# Remover colunas não numéricas, caso existam (ex: nomes dos países)
X = df.select_dtypes(include=[np.number])

```

43] ✓ 0.0s Python

2. Tratamento de outliers aplicando logaritmos com tratamento para valores negativos.

```

# PARTE 2 - 3 Os dados possuem variação alta, presença de outliers e diferença
# de escala entre variáveis
# TRATAMENTO DE OUTLIERS - Foi preciso aplicar logaritmos nas colunas que
# possuíam outliers:
# 'child_mort', 'income', 'inflation', 'life_expect', 'total_fer', 'gdp',
# 'exports', 'imports'
# verificando se o valor era negativo antes de aplicar o logaritmo

```

```

# Tratar outliers
cols_to_log = ['child_mort', 'income', 'inflation', 'life_expect', 'total_fer', 'gdp',
               'exports', 'imports']
for col in cols_to_log:
    min_val = X[col].min()
    if min_val <= 0:
        X[col] = np.log1p(X[col] + abs(min_val) + 1)
    else:
        X[col] = np.log1p(X[col])

```

✓ 0.0s Python

3. Redução da escala de valores para melhorar o resultado na aplicação dos modelos de clusterização.

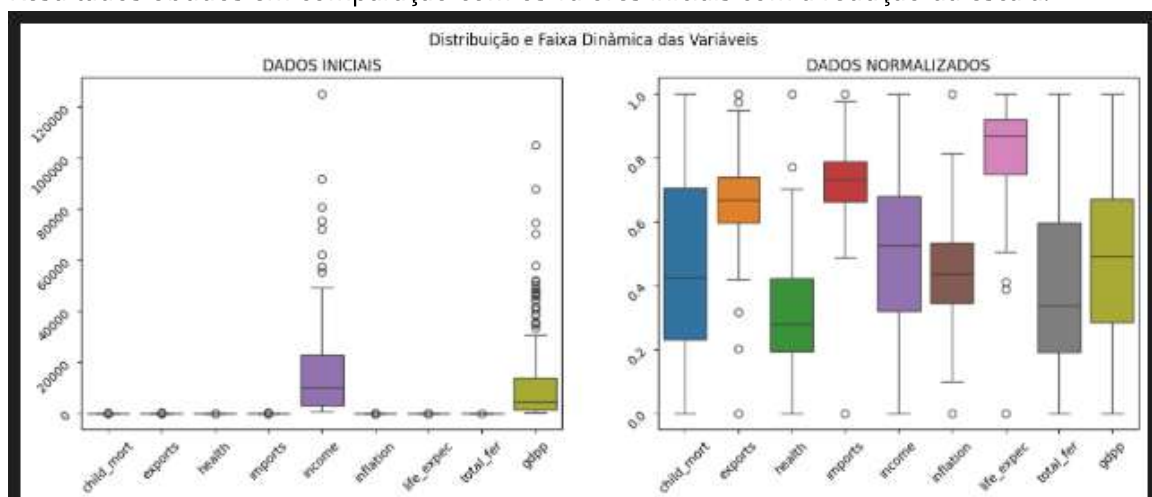
```

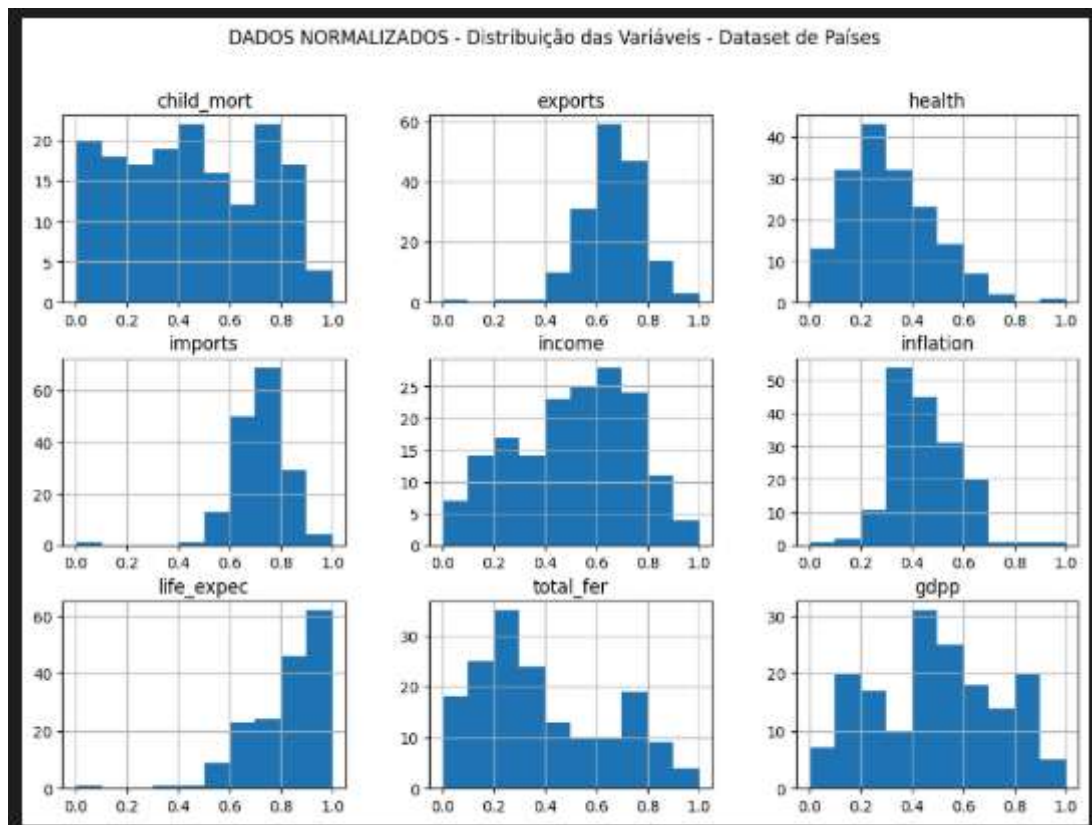
# PARTE 2 - 3 Os dados possuem variação alta, presença de outliers e diferença
# de escala entre variáveis
# NORMALIZAÇÃO - Foi preciso normalizar os dados para reduzir a amplitude
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

```

✓ 0.0s Python

4. Resultados obtidos em comparação com os valores iniciais com a redução da escala.





Parte 3 - Clusterização

Para os dados pré-processados da etapa anterior você irá:

1. Realizar o agrupamento dos dados, escolhendo o número ótimo de clusters. Para tal, use o índice de silhueta e as técnicas:

1. K-Médias

```
# ---- ESCOLHA DO K ÓTIMO PELO ÍNDICE DE SILHUETA ----
K_range = range(2, 11)
sil_scores = []

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    sil_scores.append(sil)
    print(f"k={k} -> Índice de Silhueta: {sil:.4f}")
```

25] ✓ 0.0s

```
k=2 -> Índice de Silhueta: 0.4201
k=3 -> Índice de Silhueta: 0.3251
k=4 -> Índice de Silhueta: 0.2502
k=5 -> Índice de Silhueta: 0.2584
k=6 -> Índice de Silhueta: 0.2248
k=7 -> Índice de Silhueta: 0.2217
k=8 -> Índice de Silhueta: 0.2215
k=9 -> Índice de Silhueta: 0.2086
k=10 -> Índice de Silhueta: 0.2083
```

2. DBSCAN

```
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

eps_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
min_samples = 18

print("\nTestando DBSCAN com diferentes eps:\n")

resultados = []

for eps in eps_values:
    db = DBSCAN(eps=eps, min_samples=min_samples)
    labels = db.fit_predict(X_scaled)

    # número de clusters encontrados (ignora ruído -1)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

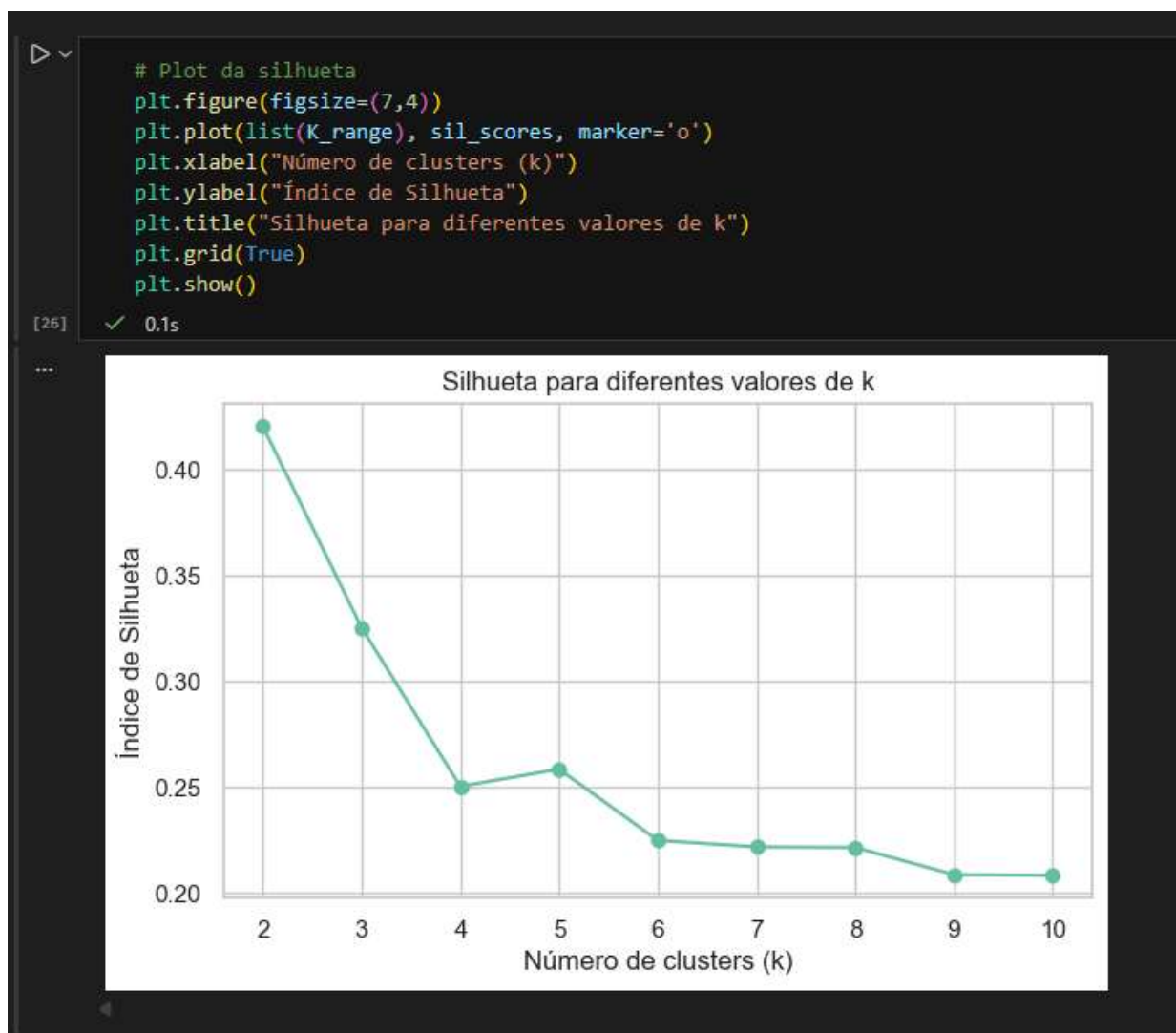
    if n_clusters > 1:
        sil = silhouette_score(X_scaled, labels)
        resultados.append((eps, n_clusters, sil))
        print(f"eps={eps} → clusters={n_clusters}, silhueta={sil:.4f}")
    else:
        print(f"eps={eps} → não formou clusters válidos")
```

✓ 0.0s

Testando DBSCAN com diferentes eps:

```
eps=0.1 → não formou clusters válidos
eps=0.2 → não formou clusters válidos
eps=0.3 → clusters=2, silhueta=0.2232
eps=0.4 → não formou clusters válidos
eps=0.5 → não formou clusters válidos
eps=0.6 → não formou clusters válidos
eps=0.7 → não formou clusters válidos
eps=0.8 → não formou clusters válidos
eps=0.9 → não formou clusters válidos
eps=1.0 → não formou clusters válidos
```

2. Com os resultados em mão, descreva o processo de mensuração do índice de silhueta. Mostre o gráfico e justifique o número de clusters escolhidos.



\

1. KMEANS - No caso do kmeans o processo utilizado na mensuração do índice de silhueta foi feita utilizando um range de 2 a 10 clusters e para cada cluster executamos o kmeans nos nossos dados e obtendo para cada caso o silhuete score. O número de clusters escolhidos para o kmeans foi 2 baseado no índice de silhueta = 0.42, olhando apenas para este índice temos o maior e melhor valor utilizando o cluster = 2.


```

# Escolhe o melhor k
best_k = K_range[np.argmax(sil_scores)]
print(f"\nMelhor k encontrado: {best_k}")
[27] ✓ 0.0s

...

Melhor k encontrado: 2

# ---- K-MÉDIAS FINAL ----
kmeans_final = KMeans(n_clusters=best_k, random_state=42)
df["kmeans_cluster"] = kmeans_final.fit_predict(X_scaled)
[28] ✓ 0.0s

print("\nClusters atribuídos (K-Médias):")
print(df["kmeans_cluster"].value_counts())
[29] ✓ 0.0s

...

Clusters atribuídos (K-Médias):
kmeans_cluster
1    92
0    75
Name: count, dtype: int64

```

\

2. DBSCAN - No caso do dbscan o processo utilizado na mensuração do índice de silhueta foi feita utilizando os valores de eps calculados previamente em um range de 0.1 a 1.0. No caso do dbscan foi realizado um teste considerando 18 vizinhos que gerou o eps de 0.3 gerou 2 clusters e apenas com esse caso obteve um resultado de silhueta de 0.22. Foi testado também a quantidade de 5 vizinhos porém o resultado foi ainda pior, a quantidade 18 vizinhos foi utilizada considerando que quando temos o `min_sample > 0` devemos utilizar: $2 \times d$, onde d são as dimensões ou features, no nosso caso temos 9 features.

```
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

eps_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
min_samples = 18

print("\nTestando DBSCAN com diferentes eps:\n")

resultados = []

for eps in eps_values:
    db = DBSCAN(eps=eps, min_samples=min_samples)
    labels = db.fit_predict(X_scaled)

    # número de clusters encontrados (ignora ruído -1)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

    if n_clusters > 1:
        sil = silhouette_score(X_scaled, labels)
        resultados.append((eps, n_clusters, sil))
        print(f"eps={eps} → clusters={n_clusters}, silhueta={sil:.4f}")
    else:
        print(f"eps={eps} → não formou clusters válidos")
```

✓ 0.0s

Testando DBSCAN com diferentes eps:

```
eps=0.1 → não formou clusters válidos
eps=0.2 → não formou clusters válidos
eps=0.3 → clusters=2, silhueta=0.2232
eps=0.4 → não formou clusters válidos
eps=0.5 → não formou clusters válidos
eps=0.6 → não formou clusters válidos
eps=0.7 → não formou clusters válidos
eps=0.8 → não formou clusters válidos
eps=0.9 → não formou clusters válidos
eps=1.0 → não formou clusters válidos
```

```
# distância ao vizinho
distances = np.sort(distances[:, n_neighbors - 1])

plt.figure(figsize=(7,4))
plt.plot(distances)
plt.title("Gráfico k-distances (estimativa do eps para DBSCAN)")
plt.xlabel("Pontos ordenados")
plt.ylabel(f"Distância ao {n_neighbors}º vizinho")
plt.grid(True)
plt.show()
```

✓ 0.2s



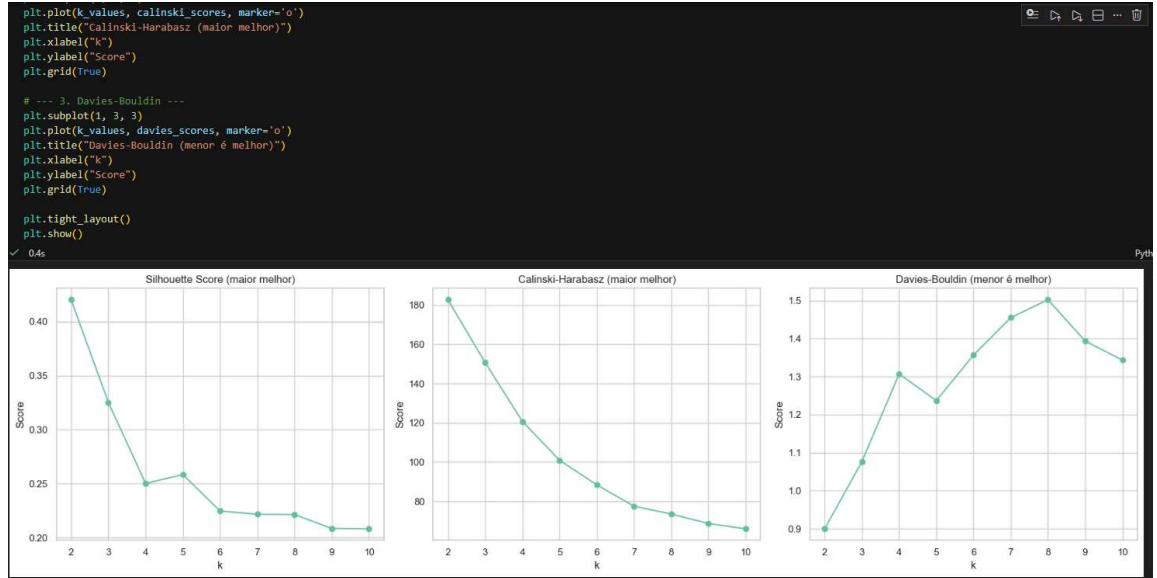
3. Compare os dois resultados, aponte as semelhanças e diferenças e interprete.

Ambos KMEANS e DBSCAN obtiveram 2 clusters como resultado apesar do DBSCAN conseguir obter apenas em uma única ocorrência o resultado de 2 clusters e não haver outras ocorrências. O Kmeans de fato consegue obter uma melhor separação dos clusters e clusters mais coesos, enquanto o dbscan, nesse caso, obteve clusters pouco separados com estrutura mais fracas pois o dataset não possui uma densidade bem marcante o que prejudicou bastante o algoritmo dbscan, com os valores baixos de eps quase todos os pontos viram ruído funcionando apenas no caso 0.3 eps.

4. Escolha mais duas medidas de validação para comparar com o índice de silhueta e analise os resultados encontrados. Observe, para a escolha, medidas adequadas aos algoritmos.

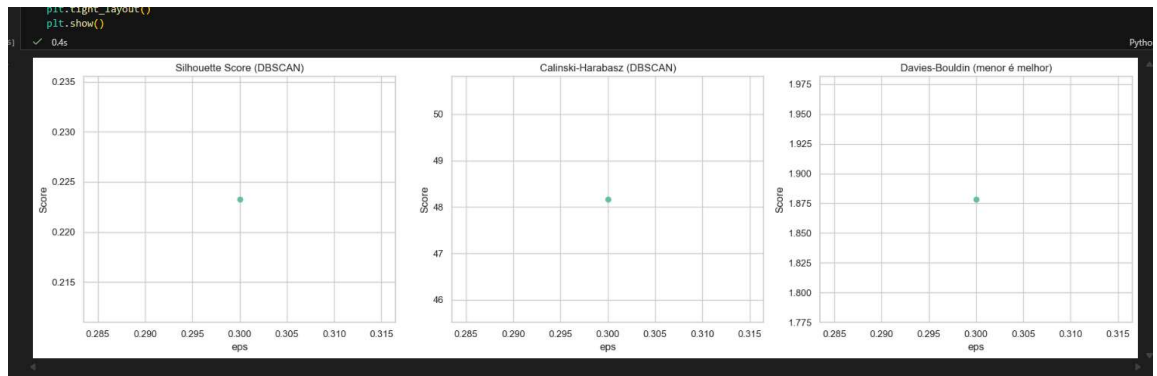
Vamos utilizar nesse caso o Davies–Bouldin Index (DBI) e o Calinski–Harabasz Index (CHI)

1. KMEANS



No caso do kmeans podemos observar que os outros dois índices adicionados DBI e CHI confirmam de fato que a utilização de clusters = 2 é de fato o melhor cenário olhando agora os 3 índices juntos Silhuete, DBI e CHI, onde o silhuete maior temos cluster = 2, no calinski temos também cluster = 2 no maior valor e no dbi também temos o ponto em cluster = 2 no menor valor do índice.

2. DBSCAN



No caso do dbscan só conseguimos apenas 1 resultado pois temos apenas o ponto de eps 0.3 gerando 2 clusters o que impactou a geração dos outros índices, no caso mantivemos o mesmo resultado anterior pois não obtivemos outras vias de comparação.

- Realizando a análise, responda: A silhueta é um o índice indicado para escolher o número de clusters para o algoritmo de DBScan?

O índice de Silhueta não é o indicador mais adequado para escolher o número de clusters no DBSCAN, pois esse algoritmo gera clusters de formas arbitrárias, possui ruídos e trabalha com densidade. A silhueta implica em clusters compactos e bem separados, o que raramente ocorre no DBSCAN. Assim, sua interpretação se torna instável e pode levar a conclusões erradas.

Parte 4 - Medidas de similaridade

- Um determinado problema, apresenta 10 séries temporais distintas. Gostaríamos de agrupá-las em 3 grupos, de acordo com um critério de similaridade, baseado no valor máximo de correlação cruzada entre elas. Descreva em tópicos todos os passos necessários.

O primeiro passo é a preparação das séries temporais para garantir que todas possuem o mesmo tamanho, frequência e não tenham valores inconsistentes, nesse momento podemos fazer normalizações e ajustes nos dados principalmente para que diferenças de escalas prejudiquem a comparação entre séries.

O próximo passo seria calcular a correlação cruzada entre cada par de séries e para cada um identificamos o maior valor de similaridade que representa o quanto são sincronizadas em algum momento.

O próximo passo é construir uma matriz que mostra o grau de semelhança entre as séries e aplicar algoritmo de agrupamento, depois essa matriz pode ser convertida para uma matriz de distancias, que já é padrão para muitos métodos de entrada. A partir dessa matriz de distancias escolhemos um algoritmo como k-medoids ou clusterização hierarquica, o algoritmo então divide as 10 séries em 3 grupos.

O próximo passo após a formação dos grupos, é fazer uma avaliação dos clusters, verificando se as séries dentro de cada grupo realmente apresentam comportamento parecido. Podemos criar visualizações, como mapas de calor ou gráficos das séries coloridas por grupo, para ajudar a interpretar os resultados. Por fim, devemos documentar toda a análise, para garantir transparência, reprodutibilidade e uma compreensão clara de como os grupos foram formados com base na similaridade entre as séries temporais.

2. Para o problema da questão anterior, indique qual algoritmo de clusterização você usaria. Justifique.

No caso do problema da questão anterior poderia ser utilizado o k-medoids pois além de utilizar um elemento real do conjunto de dados, essa abordagem é muito mais robusta e apropriada para situações em que a distância entre os elementos não é euclidiana, ele é bem robusto em relação a outliers e métricas não convencionais, por esse motivo é uma boa escolha para séries temporais baseadas em similaridade.

3. Indique um caso de uso para essa solução projetada.

Um caso de uso para utilização dessa solução poderia ser aplicada a ambientes industriais, para análise de comportamento de sensores em sistemas industriais como uma forma de manutenção preventiva.

4. Sugira outra estratégia para medir a similaridade entre séries temporais. Descreva em tópicos os passos necessários.

Uma alternativa importante para medir similaridade entre séries temporais é a técnica DTW - Dynamic Time Warping, que permite comparar séries mesmo quando elas apresentam desalinhamentos no tempo ou variações de velocidade. O processo consiste em preparar as séries, limpar e normalizá-las, calcular a distância DTW entre cada par, obtendo o custo mínimo de alinhamento, depois construímos uma matriz completa de distâncias e aplicamos algoritmos de clusterização compatíveis com esse tipo de métrica, como k-medoids ou métodos hierárquicos. Após a clusterização, os grupos são avaliados e visualizados para verificar se as séries em cada cluster realmente compartilham padrões temporais semelhantes.

Abaixo o código do notebook executado na íntegra:

```
In [1]: import kagglehub
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
from sklearn.cluster import AgglomerativeClustering
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import DBSCAN
```

```
In [2]: # PARTE 2 - 1 Baixe os dados disponibilizados na plataforma Kaggle sobre dados s
# e de saúde que determinam o índice de desenvolvimento de um país.
# Esses dados estão disponibilizados através do link:
# https://www.kaggle.com/datasets/rohan0301/unsupervised-learning-on-country-dat
path = kagglehub.dataset_download("rohan0301/unsupervised-learning-on-country-da
print(path)
# Carregar os dados
df = pd.read_csv(path+"\\Country-data.csv", sep = ",")

# PARTE 2 - 2 Quantos países existem no dataset?
print('Qtde Países no dataset: ', df['country'].count())
```

C:\Users\alan_echer\.cache\kagglehub\datasets\rohan0301\unsupervised-learning-on-country-data\versions\2

Qtde Países no dataset: 167

```
In [3]: # Visualizando as primeiras linhas e informações básicas
print("Primeiras linhas do dataset:")
print(df.head())
print("\nInformações do dataset:")
print(df.info())
print("\nEstatísticas descritivas:")
print(df.describe())
print("\nValores nulos por coluna:")
print(df.isnull().sum())
```

Primeiras linhas do dataset:

	country	child_mort	exports	health	imports	income	\
0	Afghanistan	90.2	10.0	7.58	44.9	1610	
1	Albania	16.6	28.0	6.55	48.6	9930	
2	Algeria	27.3	38.4	4.17	31.4	12900	
3	Angola	119.0	62.3	2.85	42.9	5900	
4	Antigua and Barbuda	10.3	45.5	6.03	58.9	19100	

	inflation	life_expec	total_fer	gdpp
0	9.44	56.2	5.82	553
1	4.49	76.3	1.65	4090
2	16.10	76.5	2.89	4460
3	22.40	60.1	6.16	3530
4	1.44	76.8	2.13	12200

Informações do dataset:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 167 entries, 0 to 166

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	country	167 non-null	object
1	child_mort	167 non-null	float64
2	exports	167 non-null	float64
3	health	167 non-null	float64
4	imports	167 non-null	float64
5	income	167 non-null	int64
6	inflation	167 non-null	float64
7	life_expec	167 non-null	float64
8	total_fer	167 non-null	float64
9	gdpp	167 non-null	int64

dtypes: float64(7), int64(2), object(1)

memory usage: 13.2+ KB

None

Estatísticas descritivas:

	child_mort	exports	health	imports	income	\
count	167.000000	167.000000	167.000000	167.000000	167.000000	
mean	38.270060	41.108976	6.815689	46.890215	17144.688623	
std	40.328931	27.412010	2.746837	24.209589	19278.067698	
min	2.600000	0.109000	1.810000	0.065900	609.000000	
25%	8.250000	23.800000	4.920000	30.200000	3355.000000	
50%	19.300000	35.000000	6.320000	43.300000	9960.000000	
75%	62.100000	51.350000	8.600000	58.750000	22800.000000	
max	208.000000	200.000000	17.900000	174.000000	125000.000000	

	inflation	life_expec	total_fer	gdpp
count	167.000000	167.000000	167.000000	167.000000
mean	7.781832	70.555689	2.947964	12964.155689
std	10.570704	8.893172	1.513848	18328.704809
min	-4.210000	32.100000	1.150000	231.000000
25%	1.810000	65.300000	1.795000	1330.000000
50%	5.390000	73.100000	2.410000	4660.000000
75%	10.750000	76.800000	3.880000	14050.000000
max	104.000000	82.800000	7.490000	105000.000000

Valores nulos por coluna:

country	0
child_mort	0
exports	0

```
health      0
imports     0
income      0
inflation   0
life_expec  0
total_fer   0
gdpp        0
dtype: int64
```

```
In [4]: # Remover colunas não numéricas, caso existam (ex: nomes dos países)
X = df.select_dtypes(include=[np.number])
```

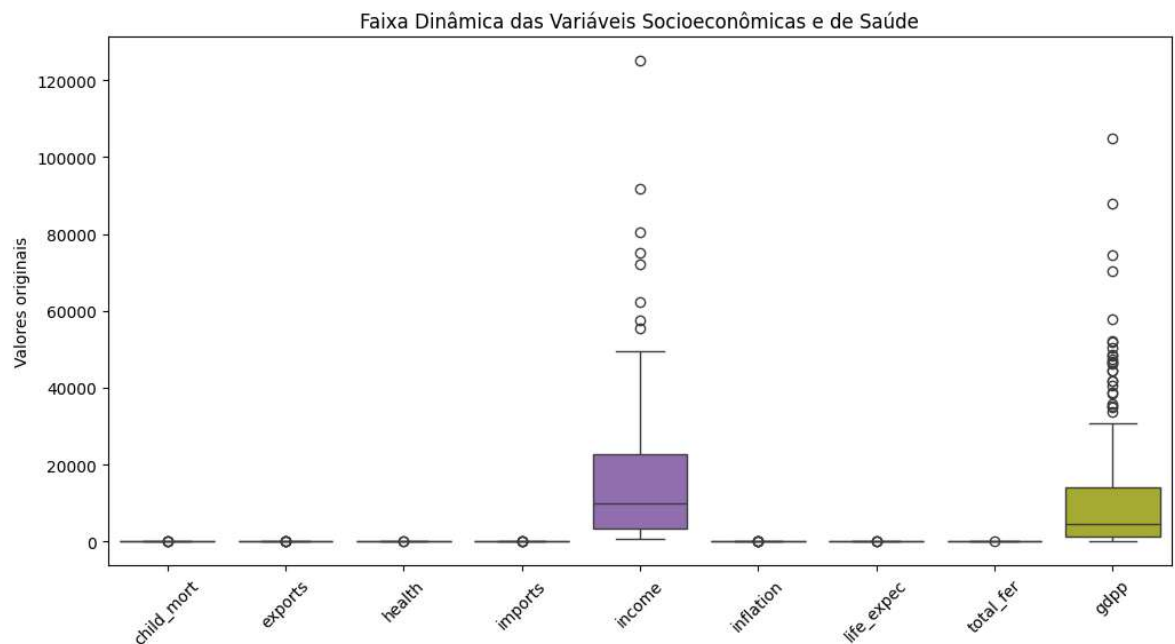
```
In [5]: # PARTE 2 - 3 Os dados possuem variação alta, presença de outliers e diferença
# de escala entre variáveis
# TRATAMENTO DE OUTLIERS - Foi preciso aplicar logaritmos nas colunas que
# possuíam outliers:
# 'child_mort', 'income', 'inflation', 'life_expec', 'total_fer', 'gdpp',
# 'exports', 'imports'
# verificando se o valor era negativo antes de aplicar o logaritmo
```

```
In [6]: # Tratar outliers
cols_to_log = ['child_mort', 'income', 'inflation', 'life_expec', 'total_fer', '
for col in cols_to_log:
    min_val = X[col].min()
    if min_val <= 0:
        X[col] = np.log1p(X[col] + abs(min_val) + 1)
    else:
        X[col] = np.log1p(X[col])
```

```
In [7]: # PARTE 2 - 3 Os dados possuem variação alta, presença de outliers e diferença d
# NORMALIZAÇÃO - Foi preciso normalizar os dados para reduzir a amplitude dos va
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [8]: X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
```

```
In [9]: # --- Boxplot para ver a faixa dinâmica ---
plt.figure(figsize=(12,6))
sns.boxplot(data=df)
plt.title("Faixa Dinâmica das Variáveis Socioeconômicas e de Saúde")
plt.xticks(rotation=45)
plt.ylabel("Valores originais")
plt.show()
```

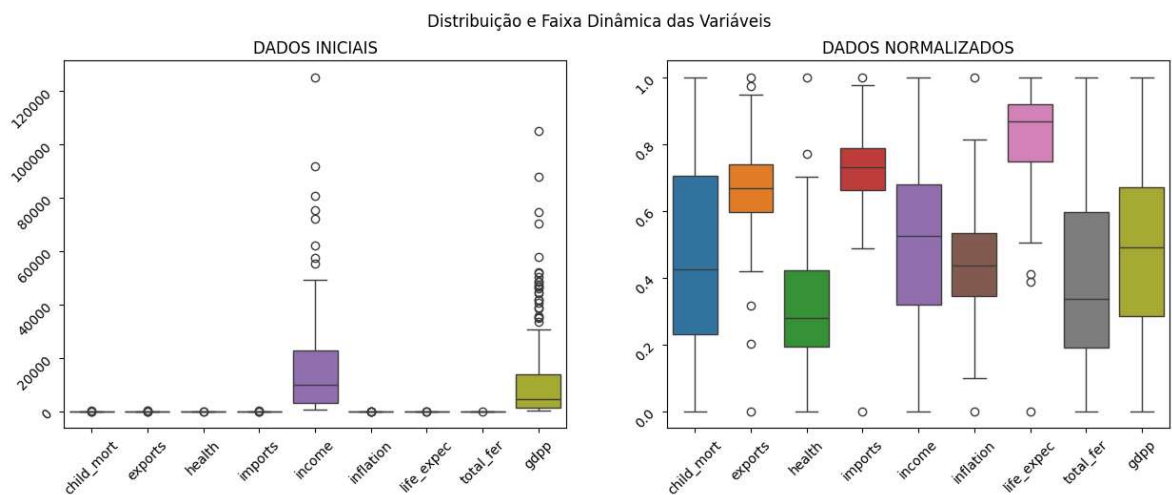



```
In [10]: f, ax = plt.subplots(1, 2, figsize=(15,5))

sns.boxplot(data=df.drop('country', axis=1), ax=ax[0])
ax[0].set_title('DADOS INICIAIS')
ax[0].tick_params(labelrotation=45)

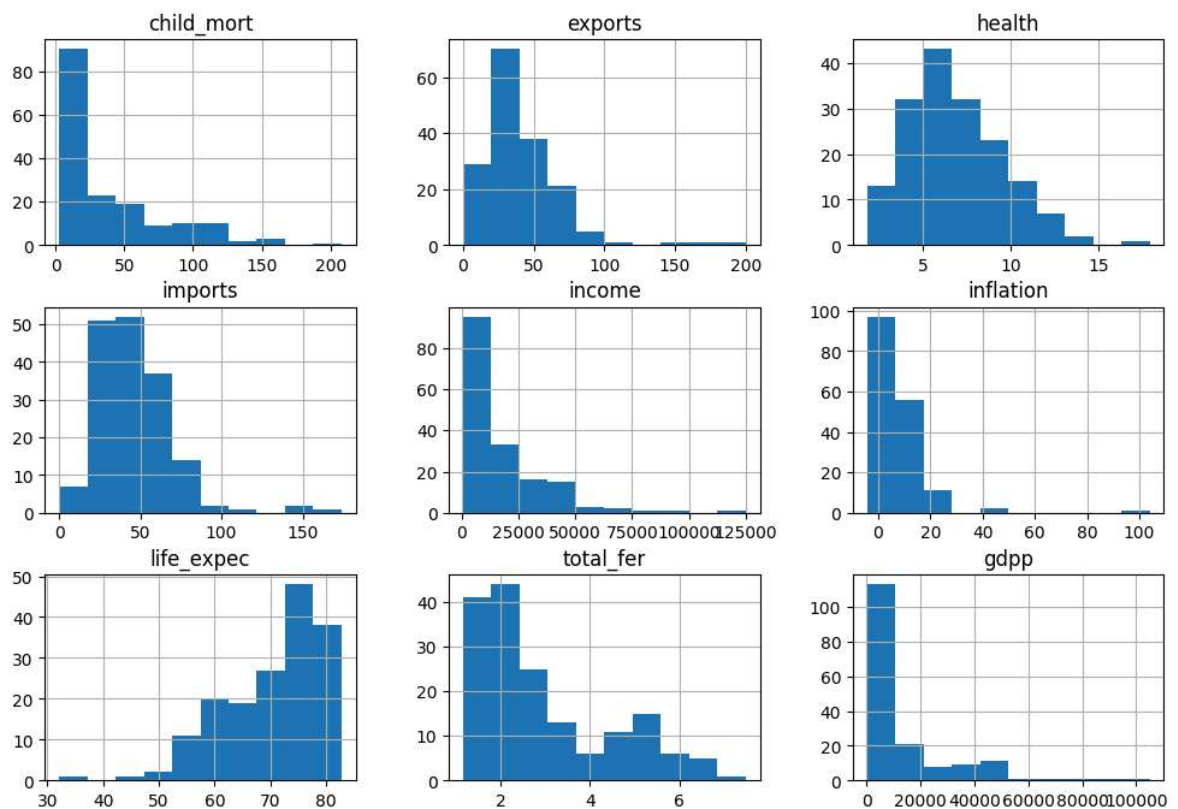
sns.boxplot(data=X_scaled_df, ax=ax[1])
ax[1].set_title('DADOS NORMALIZADOS')
ax[1].tick_params(labelrotation=45)

plt.suptitle('Distribuição e Faixa Dinâmica das Variáveis')
plt.show()
```



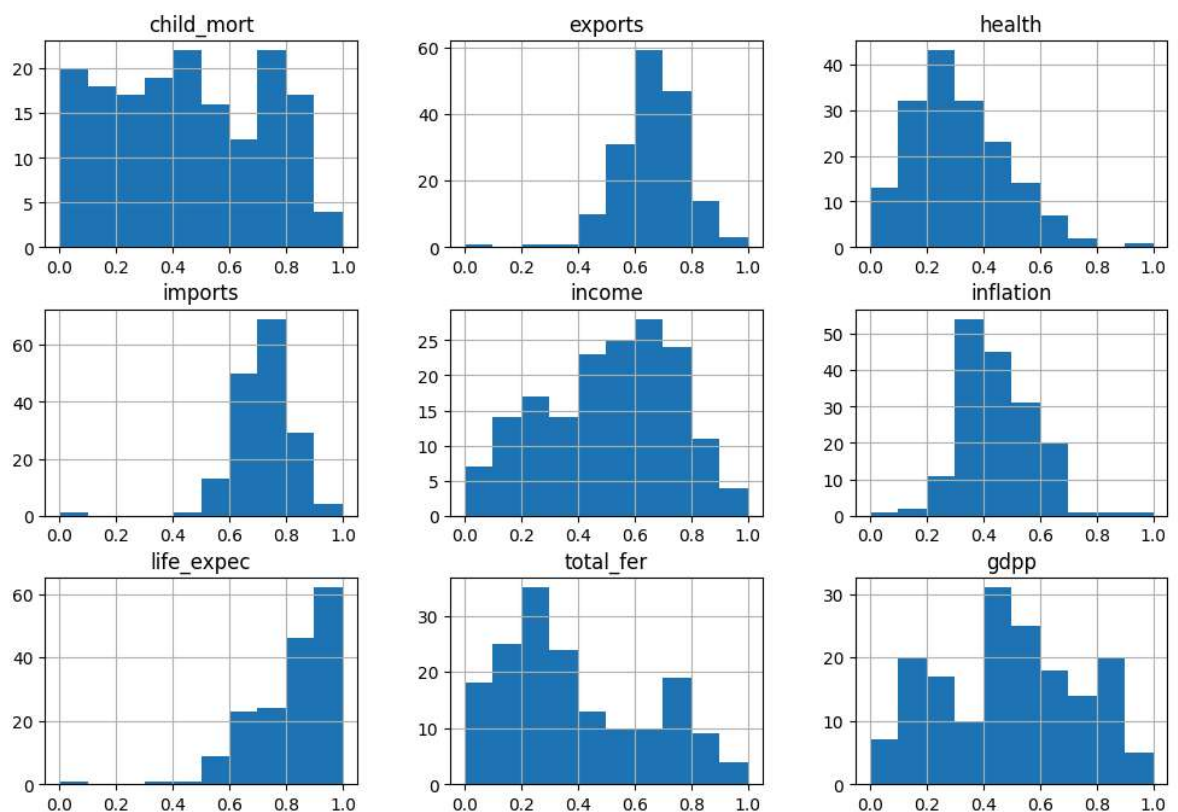
```
In [11]: df.drop('country', axis=1).hist(figsize=(12,8))
plt.suptitle("DADOS INICIAIS - Distribuição das Variáveis - Dataset de Países")
plt.show()
```

DADOS INICIAIS - Distribuição das Variáveis - Dataset de Países



```
In [12]: X_scaled_df.hist(figsize=(12,8))
plt.suptitle("DADOS NORMALIZADOS - Distribuição das Variáveis - Dataset de Países")
plt.show()
```

DADOS NORMALIZADOS - Distribuição das Variáveis - Dataset de Países



```
In [13]: number_of_clusters = 3
random_seed = 42
```

```

hierarquical = AgglomerativeClustering(
    n_clusters=number_of_clusters,
    linkage='ward',
    metric='euclidean'
)

X_scaled_df['cluster_hierarquical'] = hierarquical.fit_predict(X_scaled)
X_scaled_df['cluster_hierarquical']

```

```

Out[13]: 0      0
         1      1
         2      1
         3      0
         4      1
         ..
        162     0
        163     1
        164     0
        165     0
        166     0
        Name: cluster_hierarquical, Length: 167, dtype: int64

```

```

In [14]: kmeans = KMeans(
    n_clusters=number_of_clusters,
    random_state=random_seed
)

X_scaled_df['cluster_kmeans'] = kmeans.fit_predict(X_scaled)
X_scaled_df['cluster_kmeans']

```

```

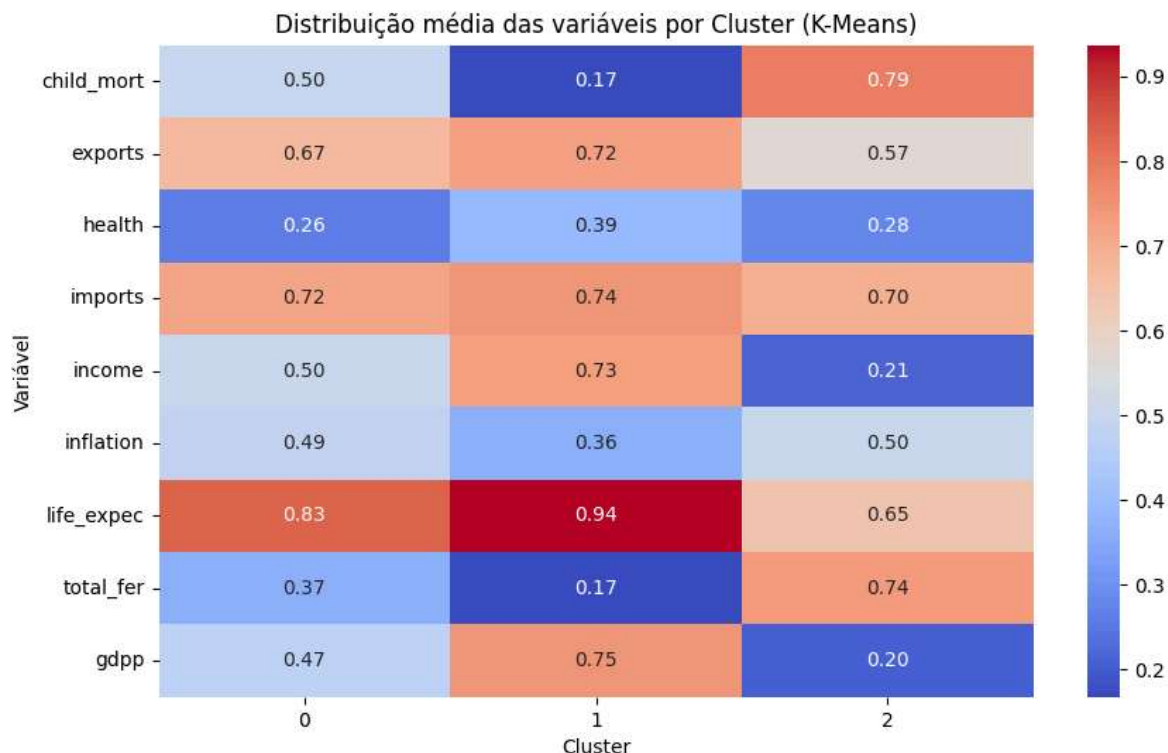
Out[14]: 0      2
         1      0
         2      0
         3      2
         4      1
         ..
        162     0
        163     0
        164     0
        165     2
        166     2
        Name: cluster_kmeans, Length: 167, dtype: int32

```

```

In [15]: # Médias por cluster (distribuição central das dimensões)
means = X_scaled_df.drop('cluster_hierarquical', axis=1).groupby('cluster_kmeans')
plt.figure(figsize=(10,6))
sns.heatmap(means, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Distribuição média das variáveis por Cluster (K-Means)")
plt.ylabel("Variável")
plt.xlabel("Cluster")
plt.show()

```



```
In [16]: # Obter os centróides
centroids = kmeans.cluster_centers_

# Calcular a distância de cada ponto ao centróide do seu cluster
distances = []
for i, row in enumerate(X_scaled):
    cluster_label = X_scaled_df.loc[i, 'cluster_kmeans']
    centroid = centroids[cluster_label]
    distance = np.linalg.norm(row - centroid)
    distances.append(distance)

X_scaled_df['distance_to_centroid'] = distances

X_scaled_df['country'] = df['country']
# Encontrar o país mais representativo de cada cluster
representative_countries = X_scaled_df.loc[
    X_scaled_df.groupby('cluster_kmeans')['distance_to_centroid'].idxmin(),
    ['country', 'cluster_kmeans', 'distance_to_centroid']]
X_scaled_df = X_scaled_df.drop('country', axis=1)

print("Países que melhor representam cada cluster:\n")
print(representative_countries)
```

Países que melhor representam cada cluster:

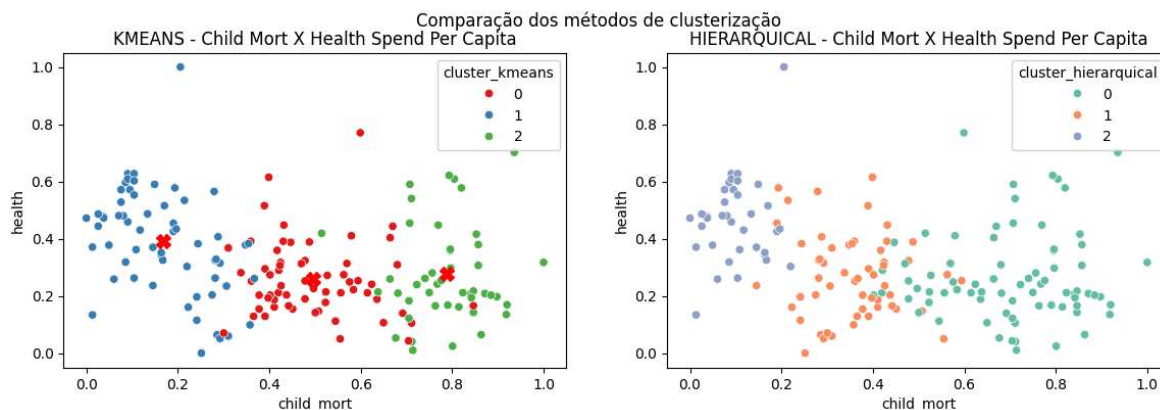
	country	cluster_kmeans	distance_to_centroid
118	Paraguay	0	0.128643
41	Croatia	1	0.145154
147	Tanzania	2	0.098071

```
In [17]: f, ax = plt.subplots(1, 2, figsize=(14,4))
x_column = 'child_mort'
y_column = 'health'
pos_for_x_column = 0
pos_for_y_column = 2
sns.scatterplot(data=X_scaled_df, x=x_column, y=y_column, hue=X_scaled_df['cluster_kmeans'])
```

```

        palette='Set1', ax=ax[0])
ax[0].set_title('KMEANS - Child Mort X Health Spend Per Capita')
ax[0].plot(
    [x[pos_for_x_column] for x in kmeans.cluster_centers_],
    [x[pos_for_y_column] for x in kmeans.cluster_centers_],
    'X',
    color='red',
    markersize=10,
)
sns.scatterplot(data=X_scaled_df, x=x_column, y=y_column, hue=X_scaled_df['cluster_kmeans'],
                palette='Set2', ax=ax[1])
ax[1].set_title('HIERARQUICAL - Child Mort X Health Spend Per Capita')
plt.suptitle('Comparação dos métodos de clusterização')
plt.show()

```



```

In [18]: print("\nDistribuição dos clusters (K-Means):")
print(X_scaled_df['cluster_kmeans'].value_counts())

print("\nDistribuição dos clusters (Hierárquico):")
print(X_scaled_df['cluster_hierarquical'].value_counts())

```

Distribuição dos clusters (K-Means):

cluster_kmeans

0 61

1 61

2 45

Name: count, dtype: int64

Distribuição dos clusters (Hierárquico):

cluster_hierarquical

0 75

1 55

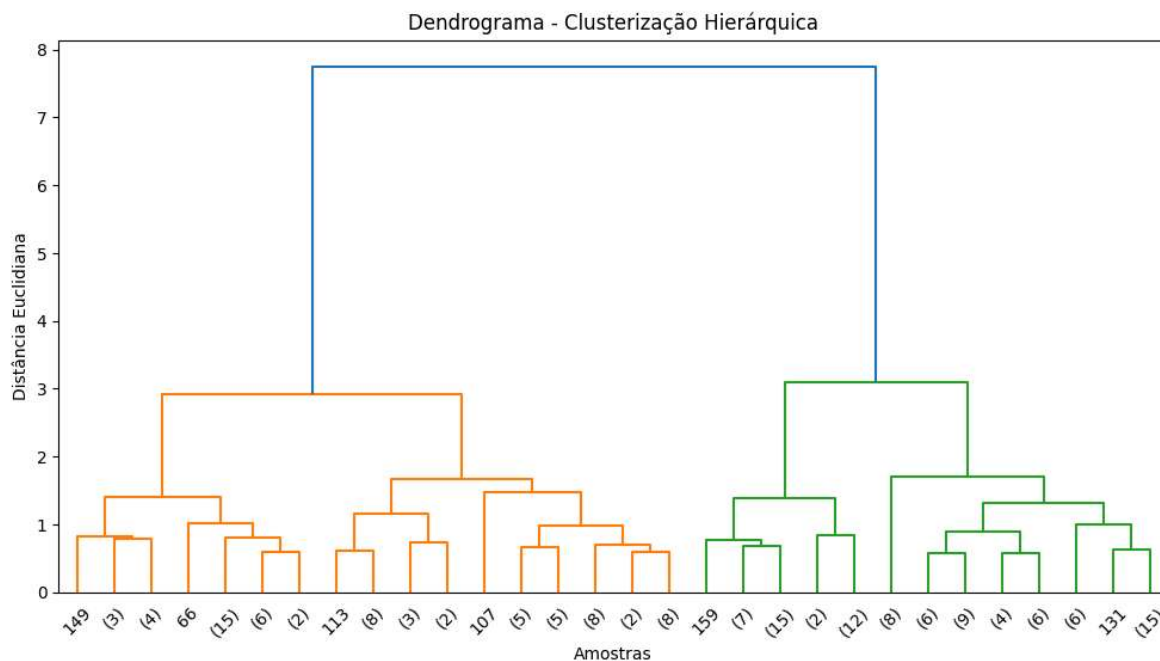
2 37

Name: count, dtype: int64

```

In [19]: Z = linkage(X_scaled, method='ward')
plt.figure(figsize=(12, 6))
dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=45., leaf_font_size=10.)
plt.title("Dendrograma - Clusterização Hierárquica")
plt.xlabel("Amostras")
plt.ylabel("Distância Euclidiana")
plt.show()

```



```
In [20]: pd.crosstab(X_scaled_df['cluster_kmeans'], X_scaled_df['cluster_hierarquical'])
```

```
Out[20]: cluster_hierarquical  0  1  2
```

cluster_kmeans

0 30 31 0

1 0 24 37

2 45 0 0

```
In [ ]: import numpy as np
import pandas as pd
from sklearn.metrics import pairwise_distances

def k_medoids(X, k, max_iter=100, random_state=42):
    np.random.seed(random_state)

    # 1 Inicialização: escolher aleatoriamente k pontos como medóides iniciais
    medoid_indices = np.random.choice(len(X), k, replace=False)
    medoids = X[medoid_indices]

    for it in range(max_iter):
        # 2 Atribuição: calcular distância de cada ponto a cada medóide
        distances = pairwise_distances(X, medoids)
        labels = np.argmin(distances, axis=1)

        new_medoids = np.copy(medoids)

        # 3 Atualização: para cada cluster, escolher novo medóide
        for i in range(k):
            cluster_points = X[labels == i]
            if len(cluster_points) == 0:
                continue

            # Calcular matriz de distâncias dentro do cluster
            intra_dist = pairwise_distances(cluster_points)
            total_dist = intra_dist.sum(axis=1)
```



```

        # Escolher o ponto que minimiza a soma das distâncias
        new_medoid = cluster_points[np.argmin(total_dist)]
        new_medoids[i] = new_medoid

    # 4 Verificar convergência
    if np.all(medoids == new_medoids):
        print(f"Convergência alcançada na iteração {it+1}.")
        break

    medoids = new_medoids

return labels, medoids

```

```

In [22]: X = X_scaled_df.drop('cluster_hierarquical', axis=1).drop('cluster_kmeans', axis=
medoids_labels, medoids = k_medoids(X, k=3)

df['cluster_medoid'] = medoids_labels
print(df.head())

```

Convergência alcançada na iteração 2.

	country	child_mort	exports	health	imports	income \
0	Afghanistan	90.2	10.0	7.58	44.9	1610
1	Albania	16.6	28.0	6.55	48.6	9930
2	Algeria	27.3	38.4	4.17	31.4	12900
3	Angola	119.0	62.3	2.85	42.9	5900
4	Antigua and Barbuda	10.3	45.5	6.03	58.9	19100

	inflation	life_expec	total_fer	gdpp	cluster_medoid
0	9.44	56.2	5.82	553	1
1	4.49	76.3	1.65	4090	2
2	16.10	76.5	2.89	4460	2
3	22.40	60.1	6.16	3530	1
4	1.44	76.8	2.13	12200	2

```

In [ ]: # -----
# 5 Visualização: comparação lado a lado
# -----

sns.set(style="whitegrid", palette="Set2")
fig, axes = plt.subplots(1, 2, figsize=(12,5))

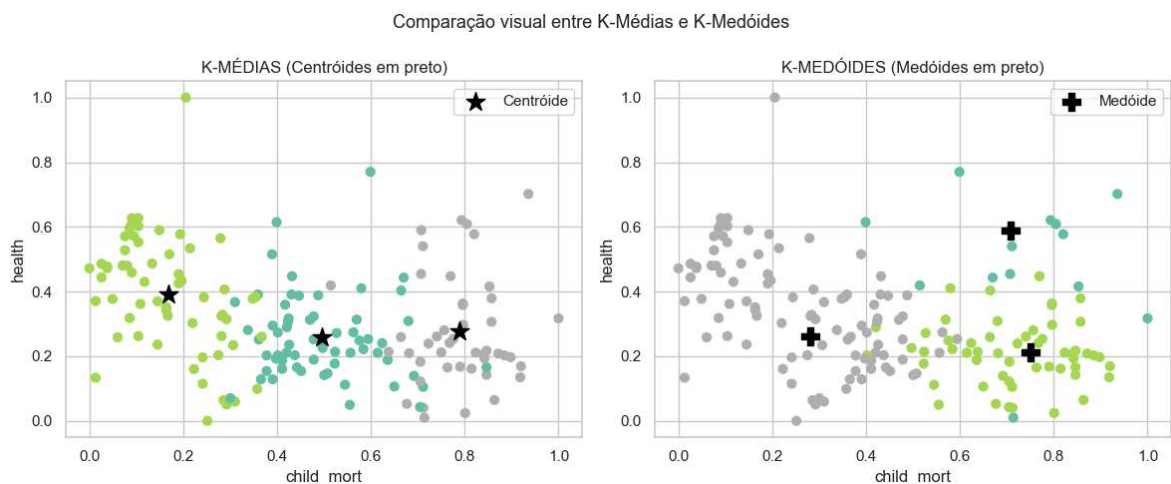
# Para facilitar a visualização, usaremos apenas duas variáveis:
x_idx, y_idx = 0, 2 # exemplo: child_mort vs health

# K-MEANS
axes[0].scatter(X_scaled[:, x_idx], X_scaled[:, y_idx], c=X_scaled_df['cluster_k
axes[0].scatter(centroids[:, x_idx], centroids[:, y_idx], marker='*', c='black',
axes[0].set_title("K-MÉDIAS (Centróides em preto)")
axes[0].set_xlabel(df.drop(columns=['country']).columns[x_idx])
axes[0].set_ylabel(df.drop(columns=['country']).columns[y_idx])
axes[0].legend()

# K-MEDÓIDES
axes[1].scatter(X_scaled[:, x_idx], X_scaled[:, y_idx], c=medoids_labels, cmap='
axes[1].scatter(medoids[:, x_idx], medoids[:, y_idx], marker='P', c='black', s=1
axes[1].set_title("K-MEDÓIDES (Medóides em preto)")
axes[1].set_xlabel(df.drop(columns=['country']).columns[x_idx])
axes[1].set_ylabel(df.drop(columns=['country']).columns[y_idx])
axes[1].legend()

```

```
plt.suptitle("Comparação visual entre K-Médias e K-Medóides", fontsize=13)
plt.tight_layout()
plt.show()
```



In [24]: *# SEGUNDA MATÉRIA VALIDAÇÃO DE MODELOS DE CLUSTERIZAÇÃO*

In [25]: *# ---- ESCOLHA DO K ÓTIMO PELO ÍNDICE DE SILHUETA ----*

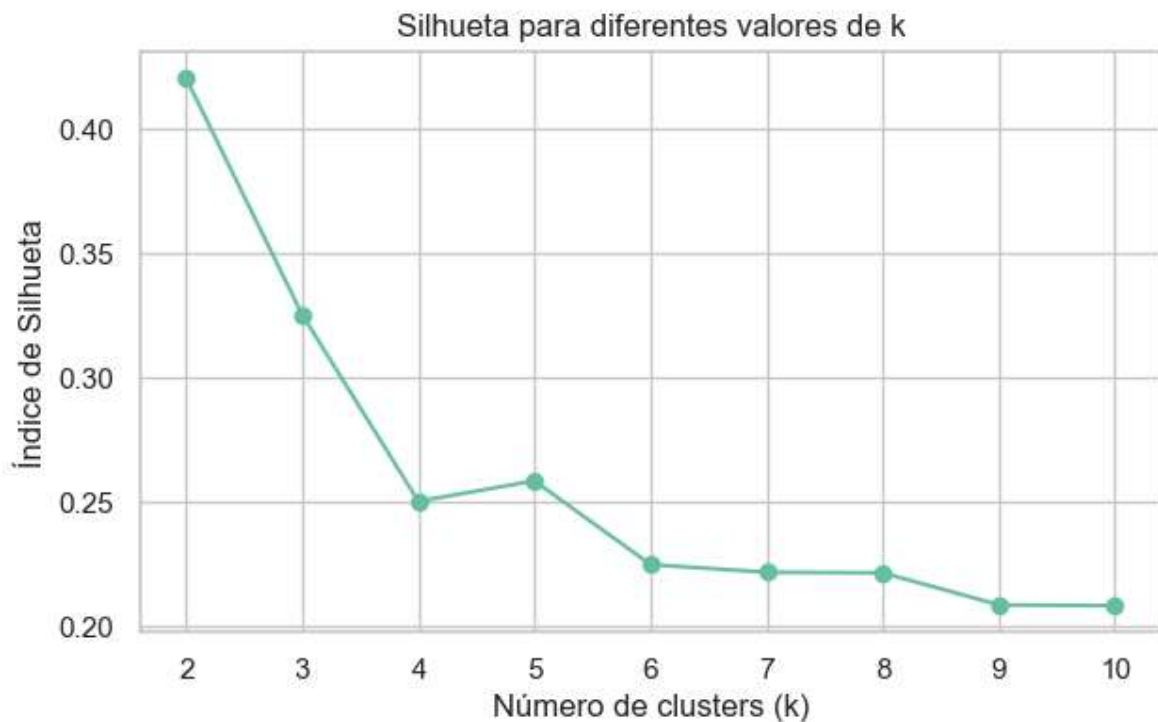
```
K_range = range(2, 11)
sil_scores = []

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    sil_scores.append(sil)
    print(f"k={k} -> Índice de Silhueta: {sil:.4f}")
```

```
k=2 -> Índice de Silhueta: 0.4201
k=3 -> Índice de Silhueta: 0.3251
k=4 -> Índice de Silhueta: 0.2502
k=5 -> Índice de Silhueta: 0.2584
k=6 -> Índice de Silhueta: 0.2248
k=7 -> Índice de Silhueta: 0.2217
k=8 -> Índice de Silhueta: 0.2215
k=9 -> Índice de Silhueta: 0.2086
k=10 -> Índice de Silhueta: 0.2083
```

In [26]: *# Plot da silhueta*

```
plt.figure(figsize=(7,4))
plt.plot(list(K_range), sil_scores, marker='o')
plt.xlabel("Número de clusters (k)")
plt.ylabel("Índice de Silhueta")
plt.title("Silhueta para diferentes valores de k")
plt.grid(True)
plt.show()
```



```
In [27]: # Escolhe o melhor k
best_k = K_range[np.argmax(sil_scores)]
print(f"\nMelhor k encontrado: {best_k}")

# ---- K-MÉDIAS FINAL ----
kmeans_final = KMeans(n_clusters=best_k, random_state=42)
df["kmeans_cluster"] = kmeans_final.fit_predict(X_scaled)

print("\nClusters atribuídos (K-MéDias):")
print(df["kmeans_cluster"].value_counts())
```

Melhor k encontrado: 2

Clusters atribuídos (K-MéDias):

kmeans_cluster

1 92

0 75

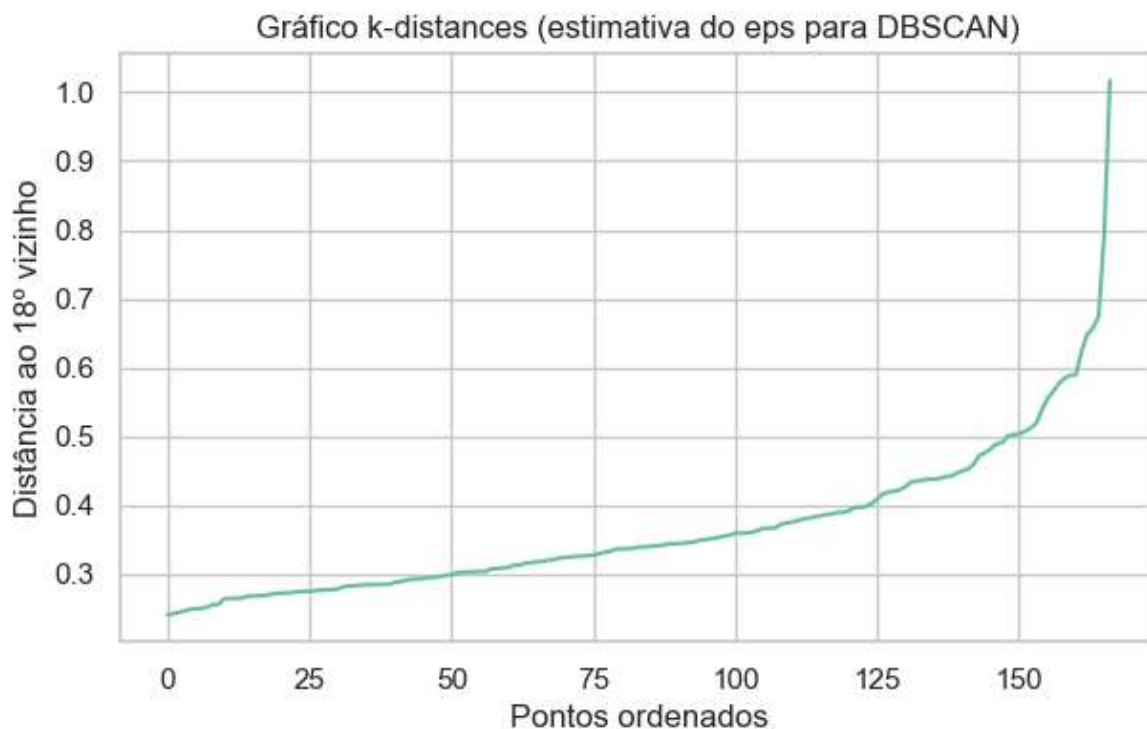
Name: count, dtype: int64

```
In [28]: # número de vizinhos usado para estimar eps
n_neighbors = 18

neighbors = NearestNeighbors(n_neighbors=n_neighbors)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)

# distância ao vizinho
distances = np.sort(distances[:, n_neighbors - 1])

plt.figure(figsize=(7,4))
plt.plot(distances)
plt.title("Gráfico k-distances (estimativa do eps para DBSCAN)")
plt.xlabel("Pontos ordenados")
plt.ylabel(f"Distância ao {n_neighbors}º vizinho")
plt.grid(True)
plt.show()
```



```
In [29]: from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

eps_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
min_samples = 18

print("\nTestando DBSCAN com diferentes eps:\n")

resultados = []

for eps in eps_values:
    db = DBSCAN(eps=eps, min_samples=min_samples)
    labels = db.fit_predict(X_scaled)

    # número de clusters encontrados (ignora ruído -1)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

    if n_clusters > 1:
        sil = silhouette_score(X_scaled, labels)
        resultados.append((eps, n_clusters, sil))
        print(f"eps={eps} → clusters={n_clusters}, silhueta={sil:.4f}")
    else:
        print(f"eps={eps} → não formou clusters válidos")
```

Testando DBSCAN com diferentes eps:

```
eps=0.1 → não formou clusters válidos
eps=0.2 → não formou clusters válidos
eps=0.3 → clusters=2, silhueta=0.2232
eps=0.4 → não formou clusters válidos
eps=0.5 → não formou clusters válidos
eps=0.6 → não formou clusters válidos
eps=0.7 → não formou clusters válidos
eps=0.8 → não formou clusters válidos
eps=0.9 → não formou clusters válidos
eps=1.0 → não formou clusters válidos
```

```
In [30]: best_eps = 0.3
best_min_samples = 18

dbscan_final = DBSCAN(eps=best_eps, min_samples=best_min_samples)
df["dbscan_cluster"] = dbscan_final.fit_predict(X_scaled)

print("\nClusters encontrados pelo DBSCAN:")
print(df["dbscan_cluster"].value_counts())
```

Clusters encontrados pelo DBSCAN:

dbscan_cluster

0	101
-1	43
1	23

Name: count, dtype: int64

```
In [31]: K_range = range(1, 11)
wss_list = []

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wss_list.append(kmeans.inertia_)

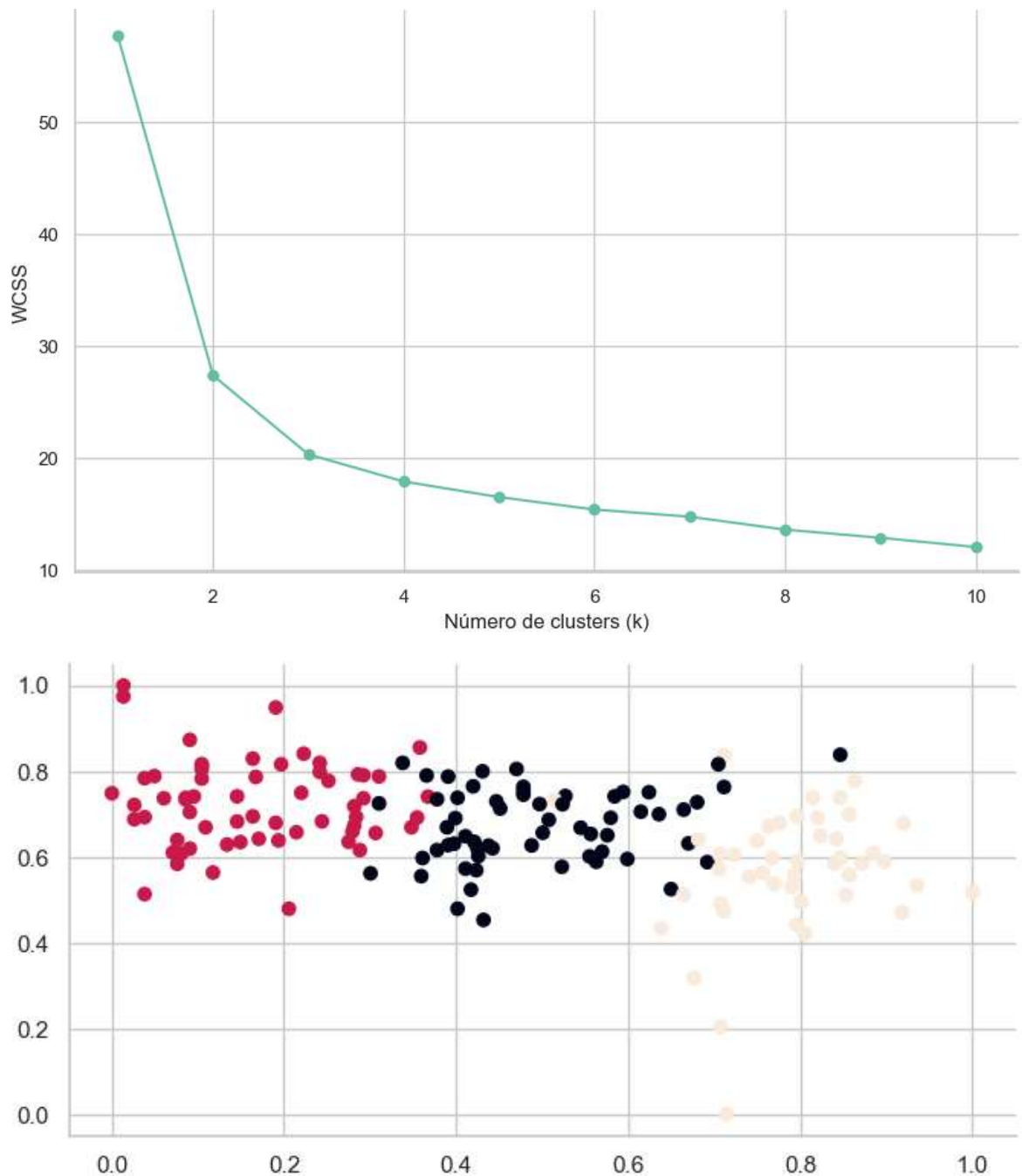
def elbow_plot(wss_list, k_init=1):
    f, ax = plt.subplots(figsize=(10,6))
    ax.plot(range(k_init, (len(wss_list)+1)), wss_list, marker='o')
    ax.set_xlabel('Número de clusters (k)')
    ax.set_ylabel('WCSS')

    sns.despine()
    plt.show()

# plot do WSS
elbow_plot(wss_list)

kmeans = KMeans(n_clusters=3, random_state=42).fit(X_scaled)

f, ax = plt.subplots(figsize=(8,4))
ax.scatter(X_scaled[:, 0], X_scaled[:, 1], c=kmeans.labels_)
sns.despine()
plt.show()
```



```
In [32]: # Faixa de k testada
k_values = range(2, 11)

silhouette_scores = []
calinski_scores = []
davies_scores = []

# ==== EXECUTAR KMEANS E CALCULAR MÉTRICAS ====
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_scaled)

    silhouette_scores.append(silhouette_score(X_scaled, labels))
    calinski_scores.append(calinski_harabasz_score(X_scaled, labels))
    davies_scores.append(davies_bouldin_score(X_scaled, labels))
```

```
In [33]: # ==== PLOT COM 3 GRÁFICOS ====
plt.figure(figsize=(18,5))
```

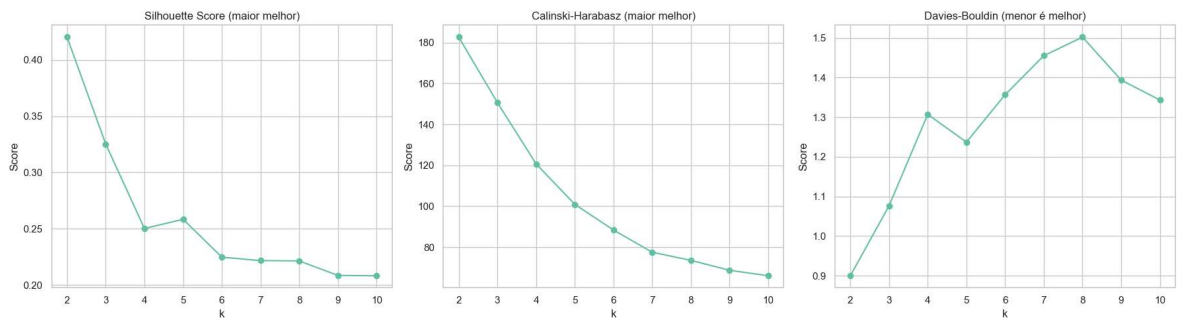


```
# --- 1. Silhouette ---
plt.subplot(1, 3, 1)
plt.plot(k_values, silhouette_scores, marker='o')
plt.title("Silhouette Score (maior melhor)")
plt.xlabel("k")
plt.ylabel("Score")
plt.grid(True)

# --- 2. Calinski-Harabasz ---
plt.subplot(1, 3, 2)
plt.plot(k_values, calinski_scores, marker='o')
plt.title("Calinski-Harabasz (maior melhor)")
plt.xlabel("k")
plt.ylabel("Score")
plt.grid(True)

# --- 3. Davies-Bouldin ---
plt.subplot(1, 3, 3)
plt.plot(k_values, davies_scores, marker='o')
plt.title("Davies-Bouldin (menor é melhor)")
plt.xlabel("k")
plt.ylabel("Score")
plt.grid(True)

plt.tight_layout()
plt.show()
```



```
In [34]: # Valores de eps a testar
eps_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
min_samples = 18

silhouette_scores = []
calinski_scores = []
davies_scores = []
valid_eps = [] # eps válido para plotar

# ==== RODAR DBSCAN ====
for eps in eps_values:
    dbscan = DBSCAN(eps=eps, min_samples=min_samples)
    labels = dbscan.fit_predict(X_scaled)

    # Número de clusters válidos (excluindo ruído = -1)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

    # DBSCAN só é válido se formar pelo menos 2 clusters
    if n_clusters >= 2:
        silhouette_scores.append(silhouette_score(X_scaled, labels))
        calinski_scores.append(calinski_harabasz_score(X_scaled, labels))
        davies_scores.append(davies_bouldin_score(X_scaled, labels))
```

```

        valid_eps.append(eps)

# ==== PLOTAGEM ====
plt.figure(figsize=(18,5))

# --- 1. Silhouette ---
plt.subplot(1, 3, 1)
plt.plot(valid_eps, silhouette_scores, marker='o')
plt.title("Silhouette Score (DBSCAN)")
plt.xlabel("eps")
plt.ylabel("Score")
plt.grid(True)

# --- 2. Calinski-Harabasz ---
plt.subplot(1, 3, 2)
plt.plot(valid_eps, calinski_scores, marker='o')
plt.title("Calinski-Harabasz (DBSCAN)")
plt.xlabel("eps")
plt.ylabel("Score")
plt.grid(True)

# --- 3. Davies-Bouldin ---
plt.subplot(1, 3, 3)
plt.plot(valid_eps, davies_scores, marker='o')
plt.title("Davies-Bouldin (menor é melhor)")
plt.xlabel("eps")
plt.ylabel("Score")
plt.grid(True)

plt.tight_layout()
plt.show()

```

